IFN647 - Machine Learning for Natural Language Processing

Assignment 1 Specification

Question 1. Parsing of Documents & Queries

Task 1.1

Define a document parsing function `docParser(stop_words, folder)` to process a collection of documents (e.g., the *Subset_RCV1v2* dataset). The parameter `stop_words` is a list of common English stop words relevant to the document collection and should be removed during preprocessing. The parameter `folder` specifies the directory containing the documents to be parsed. The function is responsible for reading the documents from the given directory and performing the required text preprocessing steps.

Each document is provided in XML format; however, the documents do not strictly conform to a single, well-defined XML schema. Therefore, you are required to design and implement a custom document parsing function, rather than relying on standard XML parsing modules (e.g., `xml.etree.ElementTree`).

The document parsing function consists of the following major steps:

Step 1) Document Reading and Processing

The function reads all files from the input `folder`. For each file, it extracts the document ID and terms, and represents each document as a `Doc` object.

You are required to define a `Doc` class using a Bag-of-Words representation with the following specifications:

- The `Doc` class must include a document identifier attribute, `docID`, which is assigned the value of the `itemid` attribute in the `<newsitem ...>` tag of the XML file.
- Each `Doc` object should be initialized with three attributes:
  - `docID`: the document identifier;
  - `terms`: an empty dictionary storing term–frequency pairs of the form *(string term : int frequency)*;
  - `doc_size`: an attribute representing the document length.
- You may define additional methods as needed, such as:
  - `get_docID()` to return the document identifier;
  - `get_termList()` to return a sorted list of all terms occurring in the document.

Step 2) Collection Representation

The function then constructs a collection of `Doc` objects for all documents. This collection may be implemented using a dictionary (as demonstrated in the workshop), a linked list, or a custom collection class. In the remainder of this specification, it is assumed that a dictionary-based structure is used, where the key is the `docID` and the value is the corresponding `Doc` object.

Step 3) Output

Finally, the function returns the complete collection of `Doc` objects.

The parsing function must implement basic text pre-processing steps, including tokenization, stop-word removal, and stemming.

Tokenization

- You are required to define the concepts of *words* and *terms* as comments in your Python implementation.
- Tokenization must be applied at least to the content within the `<text>…</text>` tags. All XML tags must be excluded, and punctuation and/or numerical tokens should be discarded according to your definition of valid words or terms.
- Implement a method `add_term()` in the `Doc` class to add a new term or increment its frequency if the term already exists in the document.

Stop-word Removal and Stemming

- Extend the provided stop-word list (`common-english-words.txt`) by incorporating additional stop words based on the characteristics of the given document collection.
- Read the stop-word file and store the stop words in a list named `stop_wordList`. When adding a term to a document, check whether it appears in `stop_wordList`; if so, the term should be ignored.
- You may apply the Porter2 stemming algorithm or any other appropriate stemming algorithm to normalize terms.

Task 1.2

Define a query parsing function `queryParser(query, stop_words)`, where the input `query` is assumed to be a simple sentence or a title provided as a string. For example, given `query = "US EPA ranks Geo Metro car most fuel-efficient 1997 car."`, the function should return a dictionary of term–frequency pairs, such as: `{'epa': 1, 'rank': 1, 'geo': 1, 'metro': 1, 'car': 2, 'fuel': 1, 'effici': 1}`.

The function must apply exactly the same text transformation and preprocessing steps as those used in document parsing, including tokenization, stop-word removal, and stemming. In particular, the tokenization rules for queries must be identical to those applied to documents to ensure consistency between query and document representations.

To validate the implementation, use following queries:

*q*1: The British-Fashion Awards
*q*2: US EPA ranks Geo Metro car most fuel-efficient 1997 car
*q*3: UK: Britain's Channel 5 to broadcast Fashion Awards.

to test the function and report the corresponding parsed outputs.

Task 1.3

Define a `main` function to test the implementations of `docParser()` and `queryParser()`. The `main` function should use the provided document collection and invoke `docParser()` to construct a collection of `Doc` objects.

For each document in the collection, the function should:

1. Print the document identifier (`docID`), the number of distinct terms, and the total number of words in the document (`doc_size`).
2. Sort the terms by their frequencies and output the resulting *`term:frequency`* list.

Finally, the `main` function should save all results to a text file named **"your full name_Q1.txt"**.

Question 1 Example Output for File "de_86042newsML.xml":
Document 86042 contains 134 indexing terms and have total 443 words
car : 9
mileag : 9
two : 8
model : 7
mpg : 7
drive : 7
sport : 7
vehicl : 7
motor : 6
light : 6
truck : 6
wheel : 6
util : 6
pickup : 5
geo : 4
fuel : 4
effici : 4
citi : 4
highway : 4
dodg : 4
wagon : 4
high : 4
sale : 4
metro : 3

top : 3
toyota : 3
epa : 3
percent : 3
four : 3
subcompact : 2
year : 2
avtech : 2
agenc : 2
rate : 2
suzuki : 2
mile : 2
per : 2
gallon : 2
…

Query: UK: Britain's Channel 5 to broadcast Fashion Awards.

The parsed query:

{'britain': 1, 'channel': 1, 'broadcast': 1, 'fashion': 1, 'award': 1}

…

Question 2. TF*IDF-based IR Model

TF*IDF is a widely used term-weighting scheme for information retrieval and NLP. It computes a weight for term *k* in document *i* using Equation (1), where the logarithm is taken with base 10. The meanings of the variables in this equation are defined in the lecture notes and should be referenced when implementing the weighting scheme.

$$d_{ik} = \frac{(\log(f_{ik})+1)\cdot\log\left(\frac{N}{n_k}\right)}{\sqrt{\sum_{j=1}^{t}[(\log(f_{ij})+1)\cdot\log\left(\frac{N}{n_j}\right)]^2}} \qquad (1)$$

Task 2.1

Define a function `df(coll)` to compute the document frequency (DF) for each term in a given `Doc` collection `coll`. The function should return a dictionary of the form `{term: df, …}`, where `df` denotes the number of documents in the collection that contain the corresponding term.

You are required to test this function and produce output consistent with the example shown below.

Example Output for Task 2.1:

There are 39 documents in this data set and contains 2245 terms
The following are the terms' document-frequency:
year : 24
report : 22
one : 21
over : 18
two : 18
day : 17
offici : 17
peopl : 17
drug : 17
more : 16
five : 15
percent : 15
state : 14
call : 13
presid : 13
three : 13
unit : 13
four : 13
month : 13
between : 12
charg : 12
citi : 12
death : 12
near : 12
open : 12
polic : 12
secretari : 12
sinc : 12
told : 12
top : 12
new : 12
number : 12
deal : 11
market : 11

…

Task 2.2

Using Equation (1), define a function `tf_idf(doc, df_, ndocs)` to compute the TF*IDF weight of each term in a `Doc` object. The parameter `doc` may be either a `Doc` object or a dictionary of the form `{term: frequency, …}`. The parameter `df_` is a dictionary `{term: df, …}` containing document frequencies, and `ndocs` denotes the total number of documents in the `Doc` collection.

The function should return a dictionary `{term: tfidf_weight, …}`, where each value represents the TF*IDF weight of the corresponding term in the given document.

Task 2.3

Define a `main` function to invoke `tf_idf()` and compute TF*IDF weights for documents in the given collection. For each document, the function should print the top 20 terms ranked by their TF*IDF weights, along with the corresponding weight values. If a document contains fewer than 20 terms, all terms should be printed.

In addition, you are required to implement a TF*IDF–based IR model with the following specifications:

- Apply the query parsing function `queryParser()` defined in Question 1 to preprocess these queries.
- For each query q, compute a ranking score for every document *doc* using the abstract ranking model defined in Equation (2), and rank the documents accordingly.

$$R(q, doc) = \sum_i (g_i(q) \cdot f_i(doc)) \qquad (2)$$

- Use three queries $q1$, $q2$ and $q3$ to test the TF*IDF–based IR model.

At last, append all outputs into text file "**your full name_Q2.txt**".

Example Output for Task 2.3:
Document 86042 contains 134 terms
mileag : 0.21382977681646675
mpg : 0.19702191524459164
truck : 0.18671234684666832
pickup : 0.174518715509528
light : 0.16122588939205384
geo : 0.15959491687686986
effici : 0.15959491687686986
highway : 0.15959491687686986
dodg : 0.15959491687686986
wagon : 0.15959491687686986
metro : 0.14035479412086554
epa : 0.14035479412086554
fuel : 0.1378100208715972
model : 0.13624610196760822
vehicl : 0.13624610196760822
motor : 0.12911675036506332
wheel : 0.12911675036506332
util : 0.12911675036506332
drive : 0.12415306150935314
sport : 0.1139285414877011

subcompact : 0.1132373641510671
avtech : 0.1132373641510671
agenc : 0.1132373641510671
gallon : 0.1132373641510671
…

The Ranking Result for query: UK: Britain's Channel 5 to broadcast Fashion Awards.
783802 : 1.051332726270421
783803 : 0.25230290149266626
741299 : 0.07862993889304021
15404 : 0.05760375906386119
80283 : 0.05019981748972497
79565 : 0.04516633877463961
86042 : 0
80884 : 0
780718 : 0
79552 : 0
37153 : 0
80484 : 0
5650 : 0
34428 : 0
3641 : 0
807606 : 0
80483 : 0
809481 : 0
…

Question 3. BM25-based IR Model

BM25 is a popular IR model with an effective ranking algorithm, which uses Equation (3) to calculate a document score or ranking for a given query *Q* and a document *doc*, where the base of *log* is *e*. You may review lecture notes to get the meaning of each variable in the equation.

$$\sum_{i \in Q} \log \frac{(r_i+0.5)/(R-r_i+0.5)}{(n_i-r_i+0.5)/(N-n_i-R+r_i+0.5)} \cdot \frac{(k_1+1)f_i}{K+f_i} \cdot \frac{(k_2+1)qf_i}{k_2+qf_i} \qquad (3)$$

Task 3.1

Define a Python function `avg_len(coll)` that calculates and returns the average document length of all documents in the collection `coll`.

- In the `Doc` class, for the attribute `doc_size` (representing the document length), implement accessor (`get`) and mutator (`set`) methods.

- You may modify the code you wrote in Question 1 to use the mutator method when creating a `Doc` object, saving the document length in `doc_size`.
- While creating the objects, accumulate the total document lengths in a variable `totalDocLength`, and at the end, compute and return the average document length.

Task 3.2

Using Equation (3), define a Python function `bm_25(coll, q, df)` to calculate BM25 scores for all documents in the collection `coll` for a given query `q`.

- The parameter `df` is a dictionary of term document frequencies in the form `{term: df, …}`.
- Parse the query `q` using the same method as for documents (you may call the `queryParser()` function defined in Question 1).
- The function should return a dictionary of BM25 scores in the form `{docID: bm25_score, …}` for all documents in the collection.

Task 3.3

Define a `main` function to implement a BM25-based IR model to rank documents in the given collection using your functions.

- Test the BM25-based IR model with queries $q1$, $q2$ and $q3$.
- For each query, print the top-6 ranked documents (sorted in descending order by BM25 score) and append the results to a text file named **`"your_full_name_Q3.txt"`**.

Example Outputs for Question 3:

Average document length for this collection is: …
The query is: UK: Britain's Channel 5 to broadcast Fashion Awards.
The following are the BM25 score for each document:
Document ID: 79565, Doc Length: 861 -- BM25 Score: 1.5332958341190779
Document ID: 86042, Doc Length: 443 -- BM25 Score: 0.0
Document ID: 80884, Doc Length: 610 -- BM25 Score: 0.0
Document ID: 780718, Doc Length: 107 -- BM25 Score: 0.0
Document ID: 783803, Doc Length: 490 -- BM25 Score: 6.0786734973597945
Document ID: 79552, Doc Length: 60 -- BM25 Score: 0.0
Document ID: 37153, Doc Length: 541 -- BM25 Score: 0.0
Document ID: 80484, Doc Length: 610 -- BM25 Score: 0.0
Document ID: 5650, Doc Length: 31 -- BM25 Score: 0.0
Document ID: 34428, Doc Length: 157 -- BM25 Score: 0.0
Document ID: 3641, Doc Length: 587 -- BM25 Score: 0.0
Document ID: 807606, Doc Length: 187 -- BM25 Score: 0.0
Document ID: 80483, Doc Length: 610 -- BM25 Score: 0.0
…

The following are the top-6 documents retrieved in response to the query -
783802 16.810554491579193
783803 6.0786734973597945
741299 2.9025753614258183
15404 2.073998609088517
80283 1.6383723697492245
79565 1.5332958341190779
…

Question 4. BM25-Based IR for RAG Prompt Generation

Retrieval-Augmented Generation (RAG) integrates information retrieval techniques with generative models to enhance prompt quality and response relevance. In this task, you will design and implement a RAG-based prompt generator using the BM25 information retrieval model defined in Question 3.

Task 4.1

Implementation of a RAG-Based Prompt Generator.

- **Input Parameters:** `q`, a user query; and `folder_name`, a directory containing a collection of documents used as the knowledge source.
- **Output:** A string containing an expanded prompt constructed by incorporating relevant contextual information retrieved from the document collection.

Your implementation must follow the procedure below:

1) **Document Retrieval** - Call the function `bm_25(coll, q, df)` to retrieve documents relevant to the query `q`.
2) **Top-k Document Selection** - Select the top-k (k = 4) ranked documents returned by `bm_25()`.
3) **Frequent Term Extraction -** From the selected top-k documents, identify the 10 most frequent words and save them in a list T. Stop words must be removed before calculating word frequencies.
4) **Prompt Construction** - Use the extracted frequent word set `T` to expand the original query `q` based on the following template:

> You are an intelligent assistant. Use the following retrieved context to answer the query.
> Retrieved Context:
> {frequent words}
> Query:
> {original query}
> Instructions: Answer the query using only the information from the retrieved context. If the answer is not contained in the context, say "I don't know". Be concise and accurate.

Task 4.2:

Test the implementation with queries $q1$, $q2$ and $q3$ to generate extended prompts which will be saved in a text file named ***"your full name_Q4.txt"***.

Optional: *You could compare the effectiveness of the generated prompts by submitting both the original queries and the RAG-extended prompts to ChatGPT (or another generative AI system).*

THE END OF ASSIGNMENT 1 SPECIFICATION